Lecture 10 — Graphs & PageRank

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

1 What is a Graph?

Core insight: A graph is a data structure made of **nodes** (entities) and **edges** (relationships between entities). Graphs are everywhere — the web, social networks, citation networks, traffic maps, and biological networks all share this structure.

Nodes (Vertices)

Represent individual entities. Examples: people in a social network, web pages, journals in a citation network, intersections in a road map.

Edges (Links)

Represent relationships between nodes. Can be **undirected** (e.g. friendship — symmetric) or **directed** (e.g. hyperlinks, citations — one-way).

Why graphs are special: The relationships between entities often contain as much information as the entities themselves. Knowing that Wikipedia is linked to by millions of pages tells you something important about its authority — something you can't see just by looking at the page content.

Graph processing tasks include:

Graph Mining

PageRank, community detection, node similarity.

Graph Learning

Node prediction, graph-level prediction (ML on graphs).

Systems

Small graphs: NetworkX, iGraph. Large graphs: Pregel, Spark GraphX, Graph DBs (Neo4j).

2 Why Graph Processing Matters

The web can be modelled as a **directed graph** where nodes are web pages and edges are hyperlinks. With billions of nodes and trillions of edges, we need scalable algorithms to extract insight from this structure.

Core problem: Not all web pages are equally important. How do we *automatically* rank billions of pages so that search engines can return the most relevant and trustworthy results?

This is the original motivation for **PageRank** — the algorithm that powered Google's early search engine. PageRank-like methods are now also used in bioinformatics (ranking proteins), social network analysis, and recommendation systems.

3 Simplified PageRank — Flow Formulation

Idea: Links as votes. If many important pages link to a page, that page must be important too. But naive in-link counting is gameable (anyone can create dummy pages). The solution: the number of "votes" a page can cast is proportional to its own importance — making the definition *recursive*.

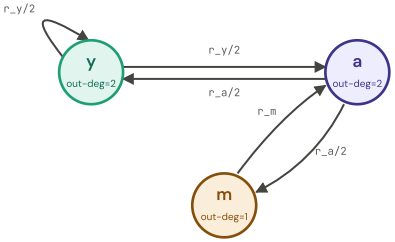
Formal definition: For page j, its rank r_j is:

```
r_j = Σ_{i→j} ( r_i / d_i )

where:  r_i = importance (rank) of page i
        d_i = out-degree of page i (number of out-links)
```

Each in-link from page i contributes a fraction r_i / d_i of i's importance. Pages with high importance and few out-links contribute the most votes per link.

Worked example — "The Web in 1839" (3 pages: y, a, m):



From the diagram, the flow equations are:

```
r_y = r_y/2 + r_a/2          # y receives half from y and half from a
r_a = r_y/2 + r_m            # a receives half from y and all from m
r_m = r_a/2                  # m receives half from a

Constraint:  r_y + r_a + r_m = 1   # scores must sum to 1

Solution:    r_y = 2/5,  r_a = 2/5,  r_m = 1/5
```

Exam trap: The system of flow equations always has a redundant equation — you must add the normalisation constraint (sum = 1) to get a unique solution. For large graphs, solving directly is impractical, so we use **Power Iteration** instead.

4 Matrix Formulation & Power Iteration

The flow equations can be written compactly using a matrix. This enables efficient computation on large graphs.

Stochastic Adjacency Matrix M

$M_{ji} = 1/d_i$ if there is an edge $i \to j$, else 0.

Each **column** represents one page's outgoing votes. Columns sum to 1 (column-stochastic). The (j,i) entry is the fraction of importance that page i sends to page j.

Rank Vector r

A vector with one entry per page. r_i is the importance score of page i. All entries sum to 1.

The flow equation becomes: **r = M · r**
i.e. r is an eigenvector of M with eigenvalue 1.

Example — M matrix for the y/a/m graph:

```
        y    a    m
    y [ 1/2  1/2   0 ]
M = a [ 1/2   0    1 ]
    m [  0   1/2   0 ]

r = M · r  is equivalent to the three flow equations above.
```

Power Iteration — how to solve r = M·r at web scale:

1. **Initialise:** Set $r^{(0)} = [1/N, 1/N, \ldots, 1/N]^T$ — all pages start with equal importance.
2. **Iterate:** Compute $r^{(t+1)} = M \cdot r^{(t)}$ — each page distributes its current importance to its out-neighbours.
3. **Stop** when the change is small: $|r^{(t+1)} - r^{(t)}|_1 < \varepsilon$. Typically converges in ~50–100 iterations.

Worked iteration for y/a/m:

Page	Iter 0	Iter 1	Iter 2	Iter 3	Converged
y	1/3 ≈ 0.333	1/3 ≈ 0.333	5/12 ≈ 0.417	9/24 = 0.375	**2/5 = 0.400**
a	1/3 ≈ 0.333	1/2 = 0.500	1/3 ≈ 0.333	11/24 ≈ 0.458	**2/5 = 0.400**
m	1/3 ≈ 0.333	1/6 ≈ 0.167	1/4 = 0.250	1/6 ≈ 0.167	**1/5 = 0.200**

Intuition: Power iteration simulates what happens if you let importance flow around the graph over and over. Each "round" every page sends its importance evenly along its out-links and receives importance from its in-links. Eventually the scores stabilise.

5 Random Walk Interpretation

Core insight: The PageRank score of a page = the long-run probability that a random web surfer is on that page. The two formulations (flow and random walk) are mathematically identical.

The random surfer model:

1. At time t=0, the surfer starts on a random page.
2. At each step, the surfer picks one of the current page's out-links uniformly at random and follows it.
3. This repeats indefinitely. Let p(t) be the probability distribution over pages at time t.
4. p(t+1) = M · p(t). As $t \to \infty$, p(t) converges to a **stationary distribution** p_s, where $p_s = M \cdot p_s$.

This is exactly the same equation as **r = M · r**. So the stationary distribution of the random walk *is* the PageRank vector. A page with high PageRank is simply a page where a random surfer spends a lot of time.

Stationary distribution = PageRank scores (for y, a, m)

6 Two Problems with Simplified PageRank

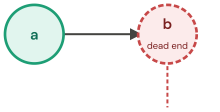
The simplified equation **r = M·r** does not always converge to something meaningful. There are two failure modes:

Problem 1 — Dead Ends

A page with **no out-links** is a dead end. The random surfer arrives but has nowhere to go. Importance **leaks out** of the system — the matrix M is no longer column-stochastic (one column sums to 0 instead of 1), so the total importance shrinks toward zero with each iteration.

Problem 2 — Spider Traps

A group of pages whose out-links all point **only within the group** (including a single page that links only to itself). The random surfer gets stuck inside and never leaves. Over time, the group **absorbs all importance** and every other page's rank goes to zero.



Dead End

Spider Trap (m absorbs all importance)

Result of both problems: Power iteration either converges to zero (dead ends) or all importance collapses into the trap (spider traps). Neither gives a meaningful ranking. We need a fix.

7 PageRank with Teleports (The Real Google PageRank)

The Google fix: At each step, the random surfer doesn't always follow a link. With probability **β** (typically 0.8–0.9) they follow a link at random. With probability **1 − β** they "teleport" to a completely random page. This prevents both dead ends and spider traps.

Fixes Spider Traps

The surfer will eventually teleport *out* of any spider trap with probability (1−β) at each step. No trap can hold the surfer forever.

Fixes Dead Ends

At a dead end, the surfer always teleports (probability 1.0). Equivalently, we add edges from every dead-end node to *every* other node in the preprocessing step.

Updated flow equation (Brin & Page, 1998):

```
r_j = Σ_{i→j} ( β · r_i / d_i ) + (1 - β) / N

# β      = damping factor (probability of following a link)
# 1 - β  = teleportation probability
# N      = total number of pages
# d_i    = out-degree of page i
```

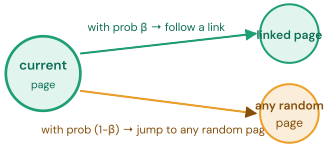
Google Matrix A — matrix form:

```
A = β · M  +  (1 - β) · [1/N]_{N×N}

# [1/N] is an N×N matrix where every entry = 1/N
# A is column-stochastic (columns sum to 1) by construction
Example: for y, a and m earlier:

            y    a    m
        y [ 1/2  1/2   0 ]             [ 1/3  1/3   1/3 ]
A = β · a [ 1/2   0    1 ] + (1 - β) · [ 1/3  1/3   1/3 ]
        m [  0   1/2   0 ]             [ 1/3  1/3   1/3 ]


PageRank equation:  r = A · r   (then solve with Power Iteration)
```



Exam tip: The damping factor β is a key design choice. Higher β → the walk trusts link structure more, converges slower. Lower β → more uniform scores, faster convergence. Common exam values: β = 0.8 or 0.85. Know how to plug into the formula and compute A.

8 Topic-Specific PageRank

Motivation: Standard PageRank measures generic importance. If you search for "Jordan", should you see basketball or the country? A sports-focused user and a geography student want different results. Topic-Specific PageRank solves this.

Key idea: Instead of teleporting to *any* random page, bias the teleport to only jump to pages within a specific **teleport set S** (a curated set of topic-relevant pages). This makes importance "flow" toward pages related to the topic.

Modified Google Matrix:

```
A_ij = β · M_ij + (1 - β) · v_i · 1^T

v_i = 1 / |S|    if page i ∈ S  # teleport only into topic set
v_i = 0          otherwise
1^T = [1]_{1×N}

# |S| = size of teleport set
# We weight pages in S equally (or can assign different weights)
```

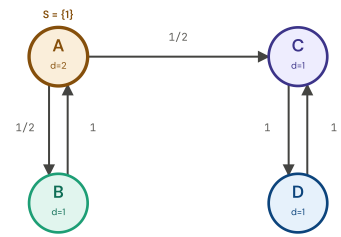
How to determine S?

- User picks from a topic menu (e.g. DMOZ 16 top-level categories: arts, sports, business, …)
- Classify the query into a topic automatically
- Use context: surrounding web page, query history, bookmarks

Effect of changing S and β

Smaller S → more concentrated importance around S.
Larger β → importance spreads further along links.
Example (node 1 is in S): S={1}, β=0.7 gives node 1 rank ≈ 0.39; β=0.9 gives ≈ 0.17 (link structure dominates more).

Worked example (from lecture) — 4-node graph, S = {1}, β = 0.8:



Edge labels = 1/d fraction sent per link. Amber node 1 is the teleport set S.

```
                A    B    C    D              v_i        1^T
          A [  0    1    0    0 ]            [ 1 ]
A_ij = β · B [ 1/2   0    0    0 ] + (1 - β) · [ 0 ] · [ 1 1 1 1 ]
          C [ 1/2   0    0    1 ]            [ 0 ]
          D [  0    0    1    0 ]            [ 0 ]
```

Iteration table (S = {1}, β = 0.8 throughout):

Node	Iter 0	Iter 1	Iter 2	…	Stable
1	0.250	0.400	0.280	…	**0.294**

2	0.250	0.100	0.160	…	**0.118**
3	0.250	0.300	0.320	…	**0.327**
4	0.250	0.200	0.240	…	**0.261**

Notice: Node 1 jumps to 0.4 in Iter 1 because 20% of all teleports land on it (1-β=0.2 flows to node 1). This boost then propagates: node 2 rises in Iter 2 because it receives from the newly-boosted node 1, and so on.

How the converged ranks shift when you change S and β:

Varying S — β fixed at 0.8

Teleport set S	r_1	r_2	r_3	r_4
{1,2,3,4} standard	0.13	0.10	0.39	0.36
{1,2,3}	0.17	0.13	0.38	0.30
{1,2}	0.26	0.20	0.29	0.23
{1}	0.29	0.11	0.32	0.26

Smaller S → more concentrated boost around the topic nodes. Node 1 rises from 0.13 to 0.29 as S shrinks to just {1}.

Varying β — S fixed at {1}

β	r_1	r_2	r_3	r_4
0.7	0.39	0.14	0.27	0.19
0.8 used above	0.29	0.11	0.32	0.26
0.9	0.17	0.07	0.40	0.36

Lower β → surfer teleports more often → node 1 gets an even larger boost. Higher β → link structure dominates → nodes 3 and 4 recover their natural authority.

Computation: Topic-Specific PageRank is computed exactly like standard PageRank — just with a different teleport distribution. The matrix is still sparse and Power Iteration still works.

9 Large-Scale Implementation — Pregel

Graph algorithms share a common pattern: each node does a local computation and passes messages to its neighbours. Pregel (Google, 2010) formalises this as the "**think like a vertex**" model.

Think like a vertex: You write a `compute()` function that describes what a *single* vertex does in a single step. The framework runs this in parallel across all vertices repeatedly until convergence.

Pregel Computational Model:

Superstep s
Each vertex v runs compute(). Reads messages from superstep s-1. May update its own value and edge values.

→

Send Messages
compute() sends messages to neighbours. These will be received in superstep s+1.

→

Vote to Halt
Vertex can deactivate itself. Woken if new message arrives. All inactive = computation stops.

Example — Computing the Maximum Value across all vertices:

```
Compute(v, messages):
    changed = False
    for m in messages:
        if v.getValue() < m:
            v.setValue(m)          # update to larger value
            changed = True
    if changed:
        for each out-neighbour w:
            sendMessage(w, v.getValue())   # propagate new max
    else:
        voteToHalt()                        # no change — deactivate
```

Each vertex broadcasts its current value. Any vertex that sees a larger incoming value updates and re-broadcasts. This continues until no vertex updates — at which point every vertex holds the global maximum.

PageRank in Pregel (C++ pseudo-code):

```
class PageRankVertex : public Vertex<double, void, double> {
  virtual void Compute(MessageIterator* msgs) {
    if (superstep() ≥ 1) {
      double sum = 0;
      for (; !msgs→Done(); msgs→Next())
        sum += msgs→Value();              // sum messages from neighbours
      *MutableValue() = 0.15 / NumVertices() + 0.85 * sum; // PageRank update
    }
    if (superstep() < 30) {
      const int n = GetOutEdgeIterator().size();
      SendMessageToAllNeighbors(GetValue() / n); // send r_i / d_i to each neighbour
    } else {
      VoteToHalt();   // stop after 30 iterations
    }
  }
};
```

Mapping to PageRank formula: Each vertex sends r_i/d_i along each out-edge. The receiver sums all incoming messages ($= \Sigma\, r_i/d_i$), then applies the update: $r_j = 0.15/N + 0.85 \times \text{sum}$. Here $\beta = 0.85$, $(1-\beta)/N = 0.15/N$.

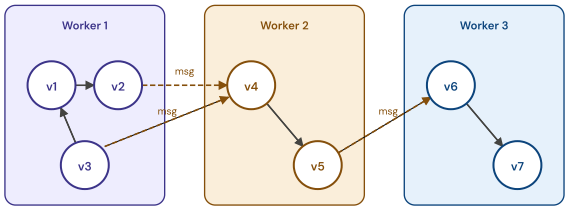
Pregel Implementation Details:

Master & Workers Architecture

Vertices are hash-partitioned (**edge cut**) and assigned to workers. Each worker keeps its portion of the graph in memory. In each superstep, workers loop through their vertices and call compute(). Messages to remote vertices are batched to reduce network traffic.

Fault Tolerance

Pregel uses **checkpointing** to persistent storage at the start of each superstep. Failures are detected via heartbeats. Failed workers are reassigned and state is restored from the last checkpoint.



▪ Solid = intra-worker graph edges · - - - → Cross-worker messages batched per superstep

10 Spark GraphX & GraphFrame

GraphX extends Spark RDDs to **Resilient Distributed Property Graphs** — graphs where both vertices and edges carry arbitrary data (properties). It supports Pregel-like distributed computation while integrating with Spark's record-oriented processing.

Property Graph

Vertices and edges each have an associated property (any type). GraphX presents multiple views: a Vertex Table, an Edge Table, and Triplets (source vertex, edge, destination vertex). You can join these like regular Spark DataFrames.

Vertex Cut Partitioning

Unlike Pregel's edge cut, Spark GraphX uses **vertex cut**: edges are partitioned across machines, and vertices are replicated as needed. This tends to work better for power-law graphs (like the web) where a few nodes have very high degree.

Spark PageRank Implementation uses two stages repeated per iteration:

Stage 1: Join

Join the ranks RDD with the adjacency list. Each edge (u, v) produces the contribution r_u / d_u for vertex v.

```
CREATE VIEW triplets AS
SELECT s.Id, d.Id, s.P, e.P, d.P
FROM edges AS e
JOIN vertices AS s JOIN vertices AS d
ON e.srcId = s.Id AND e.dstId = d.Id
```

→

Stage 2: Group-By

Group contributions by destination vertex. Sum them up and apply the PageRank formula to compute the new rank.

```
SELECT t.dstId, 0.15+0.85*sum(t.srcP*t.eP)
FROM triplets AS t GROUP BY t.dstId
```

Big picture: Spark computes PageRank with joins and aggregations instead of explicitly building and multiplying a huge matrix. The update is still the same:

$$r_i = (1 - \beta) + \beta * \Sigma_{j \to i}(r_j / d_j)$$

Stage 1 — Join (Generate Contributions)

Input tables: `vertices(Id, P)` and `edges(srcId, dstId, e.P)`, where $e.P = 1 / d_{src}$.

The join attaches source rank and edge weight to each edge. For every edge $(u \to v)$, Spark forms a triplet equivalent to $(u, v, r_u, 1/d_u)$.

Key result: one row computes one flow value $r_u * (1/d_u)$, i.e., how much rank moves from u to v.

Stage 2 — GroupBy (Aggregate + Update)

Spark groups rows by `dstId` and sums incoming flow:

$$\Sigma_{u \to v}(r_u * 1/d_u)$$

Then it applies damping/teleportation:

$$r_v = 0.15 + 0.85 * (\text{sum of incoming contributions})$$

This is exactly the usual PageRank update for each destination vertex.

Spark Step	Math Interpretation
Join	Construct all terms of the form $r_u * M_{v,u}$ on edges.
GroupBy + Sum	Compute $\Sigma_u(M_{v,u} * r_u)$ for each destination vertex v.
Add 0.15 term	Apply teleportation (constant term in this simplified form).

Tiny Example

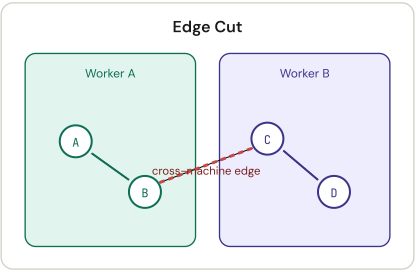
If A has rank 1.0 and two outgoing edges to B and C, then A sends 0.5 to B and 0.5 to C. B and C each collect incoming contributions, then update via `r = 0.15 + 0.85 * incoming`.

Subtle but Important

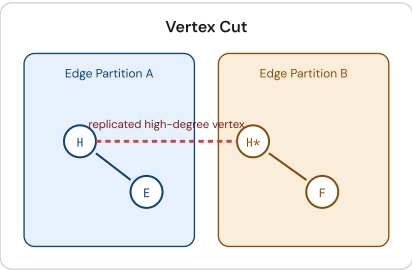
The constant `0.15` is a simplified teleport term. More generally, teleportation is `(1 - β) * v_i`, where `v_i` is the teleport distribution (uniform or topic-specific).

GraphFrame extends GraphX with a DataFrame API, making it accessible from Python and SQL, not just Scala.

Edge Cut vs Vertex Cut: Edge cut puts each vertex on one machine but cuts edges (cross-machine messages per edge). Vertex cut puts each edge on one machine but replicates vertices. For power-law graphs, vertex cut is more efficient because it avoids one super-node becoming a bottleneck.



Vertices are not replicated; cut happens on edges.



Edges are partitioned; hot vertices are replicated.

11 System Comparison

System	Paradigm	Partitioning	Language / API	Notes
Pregel (Google)	Think like a vertex, supersteps	Edge cut (hash partition vertices)	C++ (Vertex subclass)	Original BSP model; fault-tolerant via checkpoints
Giraph (Facebook)	Open-source Pregel clone	Edge cut	Java	Outperforms GraphX on trillion-scale Facebook graph (2016)
Spark GraphX	RDD-based property graph + Pregel API	Vertex cut	Scala (primary)	Achieves parity with specialised systems; integrates with Spark ecosystem
Spark GraphFrame	DataFrame-based graph API	Vertex cut	Python, Scala, SQL	Higher-level API on top of GraphX; supports Spark ML
Neo4j	Graph database + processing	Native graph store	Cypher Query Language	Best for interactive graph queries; not designed for bulk iterative algorithms

Performance finding (2016 Facebook study): Giraph outperformed GraphX on a trillion-scale social graph. GraphX is more convenient when already in the Spark ecosystem, but specialised BSP systems (Giraph, PowerGraph) can be faster for pure graph workloads.

12 Exam-Style Q&A

Q: What is PageRank and how does it differ from simply counting in-links?

PageRank recursively defines importance: a page's rank is the weighted sum of the ranks of pages linking to it, divided by their out-degrees. This means a link from an important page contributes more than a link from an unimportant page, making it hard to inflate rank by creating dummy pages (which themselves have low importance).

Q: Write the PageRank flow equation and explain each term.

$r_j = \Sigma_{i \to j} (\beta \cdot r_i / d_i) + (1-\beta)/N$. β is the damping factor (probability of following a link, typically 0.85). r_i is the current rank of page i. d_i is the out-degree of i. (1-β)/N is the teleportation term: with probability 1-β the surfer jumps uniformly to any of the N pages.

Q: What is a spider trap and how do teleports solve it?

A spider trap is a set of pages whose out-links all point within the group. A random surfer following links will get stuck inside and never leave, causing all importance to accumulate there. Teleports solve this by giving the surfer a probability (1-β) of jumping to a random page at every step, so no trap can hold the surfer indefinitely.

Q: What is a dead end and why does it cause the PageRank matrix to be non-stochastic?

A dead end is a page with no out-links. Its column in the matrix M is all zeros (sums to 0, not 1). Importance that flows into the dead end disappears from the system, causing total rank to decay to zero. Fix: treat dead ends as teleporting to every page with probability 1/N.

Q: Explain the Power Iteration algorithm for computing PageRank.

Initialise r = [1/N, …, 1/N]. Repeatedly compute $r_{new} = A \cdot r$ (where A is the Google Matrix) until $\|r_{new} - r\|_1 < \varepsilon$. Convergence is guaranteed because A is a column-stochastic matrix with positive entries (due to teleportation), so it has a unique stationary distribution. In practice ~50 iterations suffice.

Q: Explain the Pregel "think like a vertex" model. What happens in each superstep?

The user writes a compute() function for a single vertex. In each superstep, every vertex (conceptually in parallel): (1) reads messages sent to it in the previous superstep; (2) updates its own value; (3) sends messages to neighbours for the next superstep; (4) optionally votes to halt. Computation stops when all vertices are inactive (have voted to halt and received no new messages).

Q: What is Topic-Specific PageRank and how does it change the teleportation step?

Instead of teleporting uniformly to any of the N pages, the surfer teleports only to pages within a curated topic set S (e.g. all sports pages). The teleportation term becomes (1-β)/|S| for pages in S and 0 otherwise. This biases importance toward topic-relevant pages, giving different rankings per topic. One PageRank vector is computed per topic set.

Q: Compare edge cut vs vertex cut graph partitioning.

Edge cut: each vertex lives on one machine; edges crossing partitions require inter-machine messages (used by Pregel/Giraph). Vertex cut: each edge lives on one machine; high-degree vertices are replicated across machines (used by Spark GraphX). Vertex cut handles power-law graphs better because it avoids one machine becoming a bottleneck due to a super-hub vertex.

Q: What are three problems with standard PageRank and their solutions?

(1) Generic popularity — doesn't rank by topic. Solution: Topic-Specific PageRank. (2) Single importance measure — ignores other trust signals. Solution: Hubs-and-Authorities (HITS). (3) Susceptible to link spam — artificial link networks boost rank. Solution: TrustRank.